



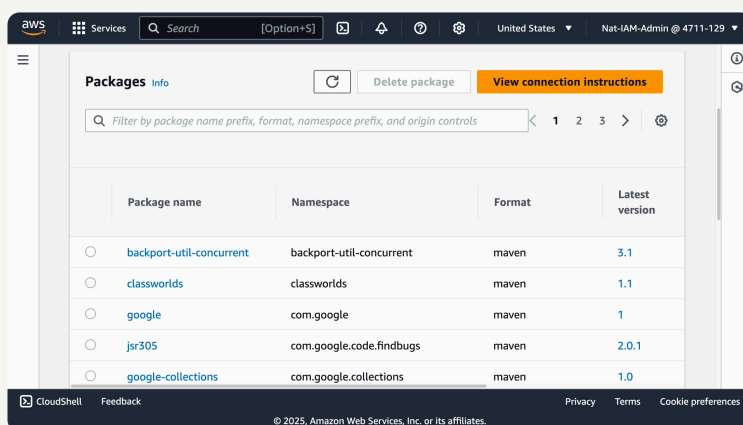
nextwork.org

Secure Packages with CodeArtifact



Muhammad Sadique

<https://www.linkedin.com/in/muhammad-sadique->





Introducing Today's Project!

In this project i will demonstrate: □ Set up CodeArtifact as a repository for your project's dependencies. □ Use IAM roles and policies to give your web app access to CodeArtifact. □ Verify your web app's connection to CodeArtifact! □ Become a package uploader - create and add your own packages to your CodeArtifact repository! CodeArtifact is an important part of a CI/CD pipeline because it makes sure an engineering team is always using the same, verified versions of packages when building and deploying the app, which reduces errors and security risks!

Key tools and concepts

Services I used were: AWS CodeArtifact Amazon EC2 Key Concepts I learnt was how to setup AWS CodeArtifact repository for your packages.

Project reflection

This project took me approximately. The most challenging part was to understand the tricky concepts. It was most rewarding to see the whole project working.

This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow.



CodeArtifact Repository

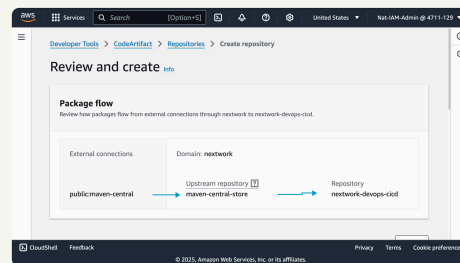
CodeArtifact is a secure, central place to store all your software packages. When you're building an application, you typically use dozens of external packages or libraries - things other developers have created that you don't want to build from scratch. An artifact repository gives you a consistent, reliable place to store and retrieve these components. Engineering teams use artifact repositories because it gives you three big benefits: 1. Security: Everyone in a team retrieves packages from a secure repository (CodeArtifact), instead of downloading from unsafe sources on the internet (hello, security risks)! 2. Reliability: If public package websites go down, you have backups in your CodeArtifact repository. 3. Control: Your team can easily share and use the same versions of packages, instead of everyone working with a different version of the same package.

A domain is like a folder that holds multiple repositories belonging to the same project or organization. We like using domains because they give you a single place to manage permissions and security settings that apply to all repositories inside it. This is much more convenient than setting up permissions for each repository separately, especially in large companies where many teams need access to different repositories. My domain is nextwork.

A CodeArtifact repository can have an upstream repository, which means to have like backup libraries that your primary repository can access when it doesn't have what you need. When your application looks for a package that isn't in your CodeArtifact repository, CodeArtifact will check its upstream repositories to find it. My repository's upstream repository is "Maven Central Store" which is essentially the App Store of the Java world. Once found, Maven will then store a copy in your CodeArtifact repository for future use.



This gives you three major benefits that you'll appreciate as your projects grow: Speed - After the first download from Maven Central, retrieving packages directly from CodeArtifact will speed up how quickly your app starts up and runs. Reliability - If Maven Central goes down (which happens more often than you'd think!), your builds keep working because you've got local copies Control - You can audit which external packages are being used





CodeArtifact Security

Issue

To access CodeArtifact we need an authorization token. Maven uses this token whenever it needs to fetch something from your CodeArtifact repository. I ran into an error when retrieving a token because my EC2 instance has no permission to AWS service called CodeArtifact. For one AWS service to access other AWS service - we need IAM Role.

Resolution

To resolve the error with my security token i created IAM policy and attach it to a role. I assign that role to my EC2 instance and that's how my EC2 got permission to access CodeArtifact.

It's security best practice to use IAM roles because they can be assume by AWS services on need basis. Permissions are permanent but role can be assumed or assigned to AWS service whenever needed.



The JSON policy attached to my role

The JSON policy I set up grants following permissions: First Statement - permissions "codeartifact:GetAuthorizationToken" allows getting an authorization token for CodeArtifact. "codeartifact:GetRepositoryEndpoint" allows retrieving the endpoint for a CodeArtifact repository. "codeartifact:ReadFromRepository" allows reading packages from a CodeArtifact repository. Second Statement - Permissions "Action": "sts:GetServiceBearerToken" allows calling the GetServiceBearerToken action from the AWS Security Token Service (STS). "Resource": "*" means this action is allowed on all resources. "Condition" adds a condition to this permission. "StringEquals" is a condition that checks for string equality. "sts:AWSServiceName": "codeartifact.amazonaws.com": This condition ensures that the sts:GetServiceBearerToken action is only allowed when the AWS service name is codeartifact.amazonaws.com. This is a security measure to restrict the use of this STS action specifically for CodeArtifact.



```
json Copy code
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "codeartifact:GetAuthorizationToken",
        "codeartifact:GetRepositoryEndpoint",
        "codeartifact:ReadFromRepository"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "sts:GetServiceBearerToken",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "sts:AWSServiceName": "codeartifact.amazonaws.com"
        }
      }
    }
  ]
}
```



Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven on how to behave across all your projects. In our case, we need a settings.xml file to tell Maven where to find the dependencies and how to get access to the right repositories (e.g. the ones in CodeArtifact). When we add CodeArtifact information to this file, we're essentially telling Maven: "Hey Maven, whenever you need to download a dependency, look in this CodeArtifact repository first, and here's how to authenticate yourself."

Compiling means translating your project's code into a language that computers can understand and run. When you compile your project, you're making sure everything is correctly set up and ready to turn into a working app.

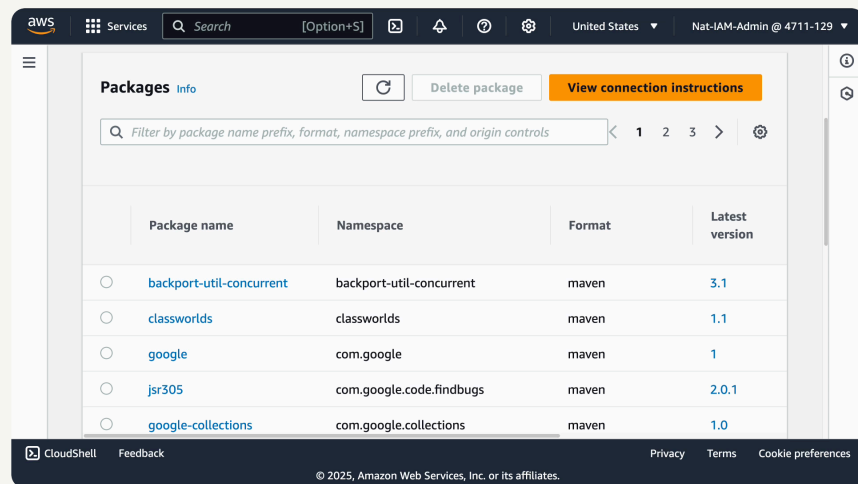


```
10 <profiles>
11 <profile>
12 <id>nextwork-nextwork-devops-cicd</id>
13 <activation>
14 <activeByDefault>true</activeByDefault>
15 </activation>
16 <repositories>
17 <repository>
18 <id>nextwork-nextwork-devops-cicd</id>
19 <url>https://nextwork-471112976395.d.codeartifact.us-east-2.amazonaws.com</url>
20 </repository>
21 </repositories>
22 </profile>
23 </profiles>
24
25 <activeProfiles>
26 </activeProfiles>
27 </settings>
```



Verify Connection

After compiling, I checked CodeArtifact repository "nextwork-devops-cicd". I noticed that packages were downloaded.



The screenshot shows the AWS CodeArtifact console. The 'Packages' tab is selected, displaying a list of packages. The table has columns for Package name, Namespace, Format, and Latest version. The packages listed are backport-util-concurrent, classworlds, google, jsr305, and google-collections. Each package has a radio button to its left, indicating it is selected. The 'View connection instructions' button is highlighted in orange.

	Package name	Namespace	Format	Latest version
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1
☐	classworlds	classworlds	maven	1.1
☐	google	com.google	maven	1
☐	jsr305	com.google.code.findbugs	maven	2.0.1
☐	google-collections	com.google.collections	maven	1.0



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

